

Introduction

This short ebook is all about writing maintainable Laravel code in general. I've been (mostly) a PHP (90% Laravel) developer since 2012 and I've been reviewing code since around 2017. I've seen (and made) some typical and often recurring issues or mistakes that make our lives harder but are usually pretty easy to fix or avoid. So I collected these issues in 3 categories:

- Performance
- Deployment
- Maintenance

The following pages contain 12 of these. I also released other high-rated materials, you can [find them here](#).

Performance Issues

The following mistakes or issues will make your app run much slower than it should be.

N+1 Queries

No matter what project I worked on in the past, N+1 queries always caused real performance issues. They usually occur only in production environment and hard to trigger them while developing. But first, let's discuss what is exactly an N+1 query problem.

Consider the following snippet:

```
class Post extends Model
{
    public function comments(): HasMany
    {
        return $this->hasMany(Comment::class);
    }
}

class PostController
{
    public function index()
    {
        foreach (Post::all() as $post) {
            $comments = $post->comments;

            // ...
        }
    }
}
```

If there are 100 posts in your database, this, first innocent-looking foreach will trigger 101 select queries. Since the comments relationship is not eager-loaded, Laravel will run an additional query for each post to get the comments associated with it. So these are the queries:

```
select * from posts
```

```
select * from comments where post_id = ?
```

The first one runs only once, but the second one runs 100 times. So we have 101 database queries. Not good. Fortunately, Laravel is a great framework and the fix is pretty easy. Instead of `Post::all()` you need to call `Post::with('comments')->get()`. It'll eager-load comments to the posts. It'll run only two database queries. One for the posts and one for the comments. You just saved 99 queries with one extra function call!

The problem is, in my experience, it's pretty hard to spot these kinds of mistakes in a code review:

```
foreach (Post::all() as $post) {}
```

vs

```
foreach (Post::with('comments')->get() as $post) {}
```

The difference in code is only a few words, but the difference in performance is **huge**.

Usually, the main source of N+1 problems is lazy-loaded relationships. But if you're inside a foreach you don't even need Eloquent relationships to cause N+1 problems. Here are a few other issues:

- Calling DB functions in loops such as `DB::table('posts')`
- Querying Eloquent model in a loop such as `Post::where('published', true)->get()`
- Saving, updating or deleting models in a loop such as `$post->save()`

Each of these requires a loop, for example:

```
public function index()
{
    foreach ($ids as $id) {
        $post = Post::find($id);
    }

    Post::all()->each(function (Post $post) {
        $post->published = true;

        $post->save();
    });
}
```

Unfortunately these kinds of N+1 problems depends more on the concrete use-case but usually they require a bit more effort than a simple `with('comments')`. Here are some general ideas:

- `find()` and `findOrFail()` accepts an array as well. So we can fix the `foreach` above: `$posts = Post::find($ids);`
- When you're updating models in a loop, usually you can run a single batch update using `->whereIn('id', $ids)`
- Sometimes you need to use a "hashmap." So instead of looping through your models and run additional queries you can do the whole thing before the loop and create a big associative array then process it.

So the above example can be rewritten such as:

```
$posts = Post::find($ids);

Post::whereIn('id', $ids)→delete();
```

You need to be extremely careful when you're dealing with large amount of data. One of constant source of bad performance at my current team is legacy Excel exports. We use the amazing laravel-excel library which provides a map function that accepts a model and you can map the data to your needs:

```
use Maatwebsite\Excel\Concerns\FromQuery;
use Maatwebsite\Excel\Concerns\WithMapping;

class InvoicesExport implements FromQuery, WithMapping
{
    /**
     * @var Invoice $invoice
     */
    public function map($invoice): array
    {
        return [
            $invoice→invoice_number,
            $invoice→user→name,
            Date::dateTimeToExcel($invoice→created_at),
        ];
    }
}
```

This is a pretty useful feature. However, the catch is that map will run **for every** invoice in your application! If you have 10 000 invoices it'll run 10 000 times when the user clicks the export button. So if you write **only one query** in the function, it'll run **ten thousand database queries**. Yep. If you write five, it'll run 50 000 queries. And you don't even need tens of thousands of records. Let's say you have only 700 invoices, but in the export you need additional data so you write six queries. The result is 4200 database query. Ouch.

Another dangerous and hard-to-see form of N+1 query is notifications. For example, this is the "free trial about to expire" notification of Laracheck:

```
public function toMail($notifiable)
{
    $checkResults = $notifiable->allCheckResults();

    return (new MailMessage)
        ->subject('Your Trial Ends Tomorrow')
        ->markdown('mail.free-trial.expires', [
            'user' => $notifiable,
            'checkResults' => $checkResults,
            'issuesFound' => $checkResults
                ->sum(fn (CheckResult $check) => $check-
>numberOfIssues()),
        ]);
}
```

Each time this notification is sent out it'll call the `allCheckResults()` method which will run a few database queries. Let's say three. And once again, numbers add up quickly. If you have 5000 users it's 15 000 queries.

So these are some common occurrences of N+1 problems:

- Lazy-loaded relationships.
- Database queries in loops or Collection methods. Nested loops are even more dangerous.
- Database queries in "indirect loops." By indirect loop, I mean something like `toMail()` in Notifications or `map()` in exports. Methods that will be invoked multiple times, usually by the framework.

You need to be absolutely careful about these situations because they cause real production issues. Fortunately, Laracheck can detect N+1 queries so you don't need to worry about them.

Missing `whenLoaded()` in resources

Laravel HTTP Resources are great! However, they can be a source of poor performance. Consider this class:

```
class PostResource extends JsonResource
{
    public function toArray($request)
    {
        return [
            'title' => $this->title,
            'content' => $this->content,
            'comments' => CommentResource::collection(
                $this->comments
            ),
        ];
    }
}
```

Nice and simple. Now let's take a look at how we use it in a Controller that returns a list of posts:

```
class PostController
{
    public function index()
    {
        return PostResource::collection(Post::all());
    }
}
```


Nothing crazy so far. However, this API endpoint (or web route) will perform N+1 number of database queries. First, it queries the posts (`Post::all()`) then it'll create a Resource for each one. In the `PostResource` class we perform a `select * from comments where post_id = ?` query. Since this will run for every post it introduces an N+1 problem in your application.

We can solve this problem in two steps:

- Only access the comments relationship in the Resource if it's already loaded in the model
- Eager-loading the comments relationship in the controller using `Post::with('comments')->get()`

This is what the new resource looks like:

```
class PostResource extends JsonResource
{
    public function toArray($request)
    {
        return [
            'title' => $this->title,
            'content' => $this->content,
            'comments' => CommentResource::collection(
                $this->whenLoaded('comments')
            ),
        ];
    }
}
```

The only difference is the `whenLoaded()` call but it makes a big difference! If the comments relationship in the Post model hasn't been already loaded it will not access the relationship and will return an empty Collection.

So to make it work we need to change the controller as well:

```
class PostController
{
    public function index()
    {
        return PostResource::collection(
            Post::with('comments')->get()
        );
    }
}
```

Now that comments are eager-loaded the Resource will include them in the resulting array. And now the API has only two database queries:

```
select * from posts
select * from comments where post_id IN ( ... )
```

And that's it! Now this API has a much better performance and runs much less database queries.

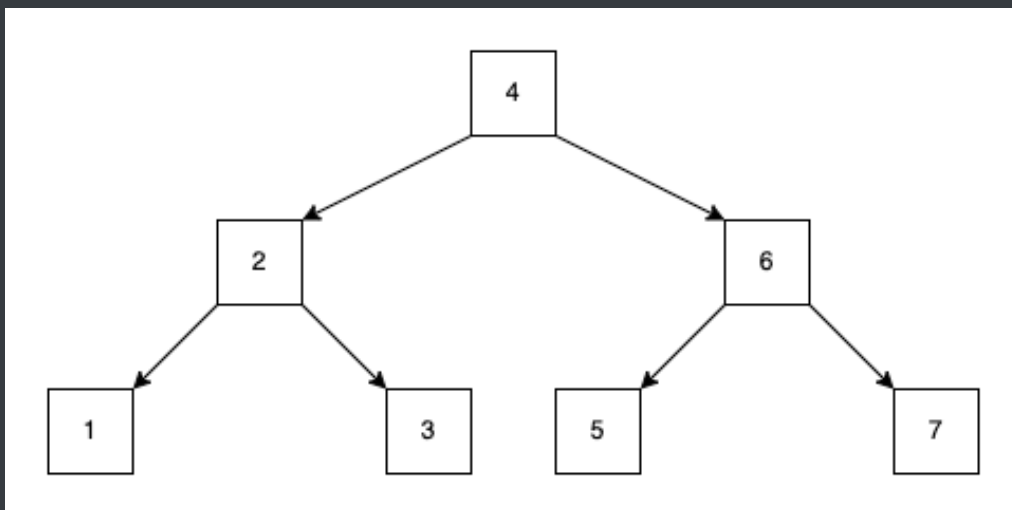
Fortunately, if you're using [Laracheck](#) you don't have to worry about these kinds of mistakes! It'll take care of these and warn you about missing `whenLoaded()` calls.

Missing database indices in migrations

One of the main reasons for slow database queries is missing or wrong indices. So first, we need to understand what is an index. Under the hood, MySQL stores tables in files. So every time you make a query it needs to fetch the data from the filesystem. Which is slow. This is why indices exist.

When you add a B-TREE (balanced tree) index to your table MySQL will load the column's content into a tree data structure and it doesn't have to fetch all the rows in a sequential way. And this tree is ordered. This is important since searching in an ordered dataset is much more efficient than searching in an unordered dataset. Just think about binary search, for example.

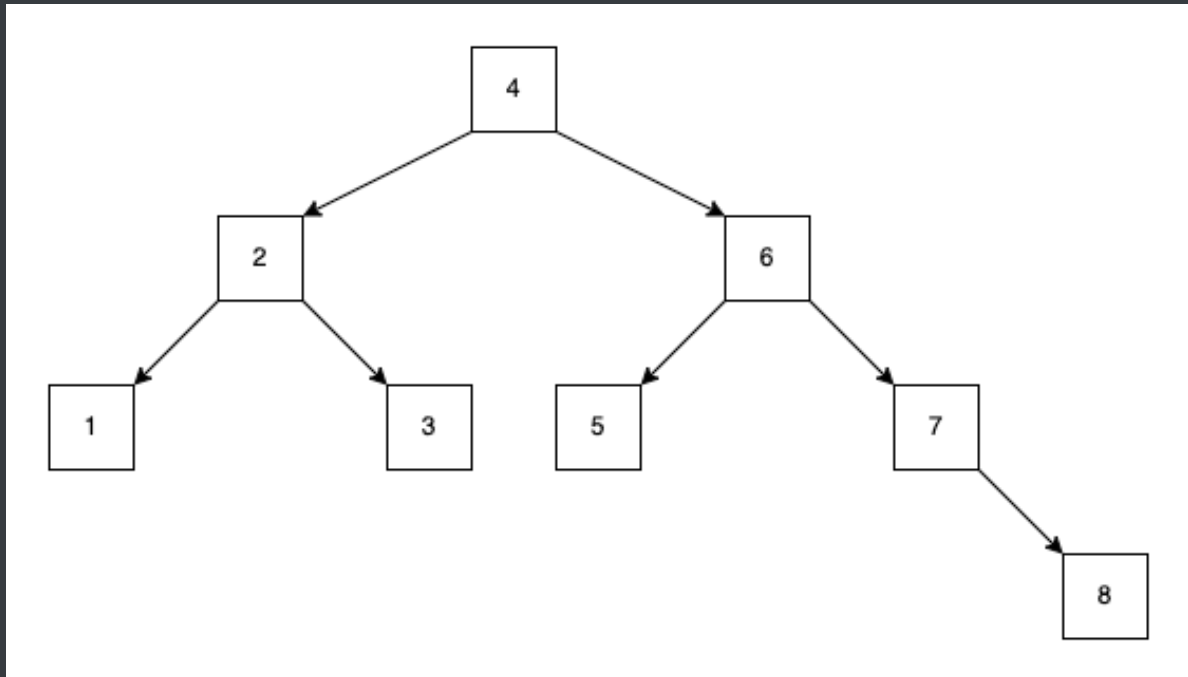
Here's a balanced tree:



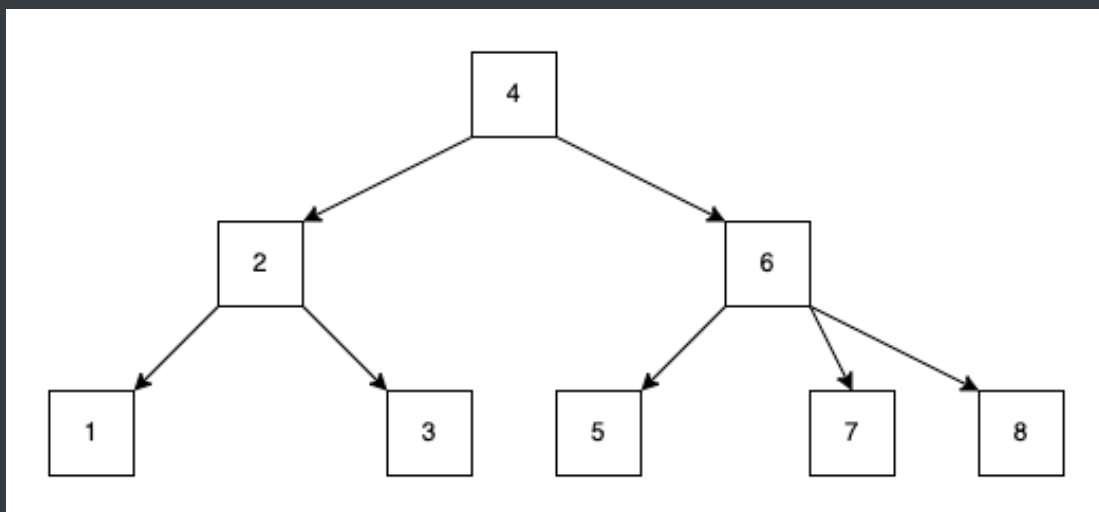
It has three pretty important rules:

- The **left** subtree always contains nodes that are **less than** the root node. For example, 2 and 4, 1 and 2, 5 and 6
- The **right** subtree always contains nodes that are **greater than** the root node. For example, 6 and 4, 7 and 6 or 3 and 2
- The leaf nodes always has the same depth. You can see a 3-level B-TREE in the image where each subtree has the same depth. This is no coincidence. This is why it's called **balanced**.

If you want to insert the number 8 into this tree, you cannot do that:



This tree is not balanced. The most right subtree is deeper than the others which means it's finding number 8 takes more iteration and more time. To keep the tree balanced we need to do something like this:



Remember, the "B" stands for "balanced" not "binary"!

All right, now back to reality. Gimme some Laravel code:

```
public function up()
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();

        $table->string('title');
        $table->string('body');

        $table->dateTime('published_at')->index();

        $table->timestamps();
    });
}
```

When you add an index such as `published_at` MySQL will store the dates in the B-TREE itself (in memory). Which makes queries pretty fast, because the DB engine can search in the sorted, balanced tree compared to fetching everything in a linear fashion from the filesystem. Indexing is a pretty complex topic but these are the basics. If you miss an important index it can literally set your servers on fire.

If you want to learn more about indexing, I recommend [this amazing Laracon talk](#).

Deployment Issues

The following mistakes or issues will make your deployment or rollback a bit (or way) harder.

Missing `down()` method in migrations

When a deployment goes wrong and you need to roll back to a previous version it's crucial to have this process as easy and fast as possible. The `down` method in your migration plays a **colossal** role in deployment rollbacks. If you miss one your app will be left in an inconsistent state.

For example, let's say you have a legacy `Post` in your application that looks like this:

id	title	type
1	Office Party	event
2	Don't forget to upload your profile picture	notification
3	Introducing our newest developer	news

Each post has a `type` which determines the post's behavior. For example, an event shows up in your calendar, a notification will annoy you when you log in to the app and so on.

I'm not saying that storing strings as types is a bad thing, but let's say our requirements force us to refactor this table. We'd like to introduce a new model for the types. Maybe this way, it's easier to implement unique behavior to the different post types.

So we create a new table and update the posts:

id	title	post_type_id
1	Office Party	1
2	Don't forget to upload your profile picture	2
3	Introducing our newest developer	3

id	name
1	event
2	notification
3	news

After this refactor you probably have a separate class for each post type that implements the strategy pattern. Something like that:

```
interface PostType {}

class Event implements PostType {}
class Notification implements PostType {}
class News implements PostType {}
```

Or you can introduce enums as well. It doesn't really matter for our purpose, the point is, with the new DB structure you have a more flexible application.

So you wrote your migration, added the new models, some strategy classes, and enums and you deploy it to production. But your application breaks. And it has a big impact on users and seems quite complicated, so instead of releasing a hotfix, you roll back to the previous version. The version that didn't have the `PostType` models and the separate table. But you realize that your migration doesn't have a `down()` method. It looks something like this:

```
public function up()
{
    Schema::create('post_types', function (Blueprint $table) {
        $table->id();

        $table->string('name');
    });

    foreach (Post::all() as $post) {
        /**
         * migrate the string values from the post
         * and update the relationships
         */
    }
}

public function down()
{
    //
}
```

So your application is in an inconsistent state:

- Your code assumes there's a `type` property in the `Post` model that contains a string value
- But in your database there's no `type` column but a `post_type_id` integer foreign key and a `post_types` table

The point is: there's no such thing as smooth rollback without `down()` methods in your migrations. The vast majority of the time it won't be a big problem since you probably have a lot of migration that creates a new table and drops it in the `down` method. However, when it comes to refactoring table from structure A to structure B and migrating old data, you need to write the `down` method yourself. And often these kinds of releases are the most riskiest.

Missing env variables

Lots of Laravel deployments and pipelines start with this:

```
cp .env.example .env
```

Also, every new developer who joins the team starts with this command. This is why it's a good idea to add every new env variable to the `.env.example` file as well. It's a small thing but it makes deployments and onboarding just a little bit easier. And it doesn't take too much effort.

Inconsistent composer files

If you want to install a new package or update an existing one it's possible that you run the wrong `composer` command and accidentally update every package. This is why I always check the contents of both `composer` files when someone modified it.

Another problem can be when you changed `composer.json` but didn't changed `composer-lock.json`. This is a bad idea, since most deployment will install packages from the lock file to make sure production has the exact same dependencies as your local environment.

If you're not sure about the lock file here's a quick summary:

- You install a package using `composer install laravel-excel`
- Composer will download the package, update the `composer.json` file and also the `composer-lock.json`
- In `composer.json` you'll see something like this: `"laravel-excel": "^1.2"` This means that project **some** version of laravel-excel `>=1.2.0` and `<2.0.0`
- Let's say you installed a package when the newest version 1.2.1, but after 6 months you run `composer update`. According to your version in the JSON file it might update to package to 1.9.0 if such a version exists.
- If you think about that, it's pretty dangerous. Imagine if every new deployment would install the newest version of the package. Without you ever knowing it... I mean, it wouldn't be so dangerous in a perfect world where every package developer follows the core concepts of semantic versioning. However, I highly doubt that. This is why composer introduced the lock file.
- If you install a new package, and take a look at the `composer-lock.json` file you will see something like this:

```
{  
  "name": "rollbar/rollbar-laravel",  
  "version": "v7.2.0",  
  "source": {}  
}
```

- As you can see this version number doesn't contain things like `^` or `*` or anything like that. It's an **exact** version number. So in this file, we lock the dependencies to an exact version. The ones we're using while developing.
- After that you push the lock file into your git repo. This is important, because every new developer will pull it and run `composer install`. If a lock file is present then composer will use that file, instead of the json. So the new dev will have the exact same dependencies as you.
- And this point also applies to deployments. Each deployment will install the exact versions from the lock file.

This is why I always another look at the composer json and lock files whenever someone (for example, me) modified them.

env() calls outside of config files

In Laravel, it's a best practice to use `env('MY_ENV_VALUE')` calls only in config files. There are two reasons.

Often config values are cached in a production environment using the `php artisan config:cache` command. If you don't know about this command, you should consider using it. It'll cache every config file into memory. So whenever you use them with `config('app.my_value')` it'll retrieve the value from memory instead of touching the `.env` file on the disk.

If you have `env()` calls in your code (outside of config files), this config caching can break your production environment! Or at least it can cause bugs.

The other reason is that `config` values can be "mocked" in tests pretty easily. All you have to do is this:

```
class ListProductsTest extends TestCase
{
    use RefreshDatabase;

    /** @test */
    public function it_should_use_the_default_page_size()
    {
        config(['app.default_page_size' => 10]);

        $products = $this->getJson(route('products.index'))
            ->json('data');

        $this->assertCount(10, $products);
    }
}
```


Maintenance Issues

The following mistakes or issues will make your life just a bit harder in general.

Missing foreign keys in migrations

MySQL foreign keys are a great way to ensure data integrity in your database. If you have a foreign key it'll prevent you to reference an invalid ID from another table. For example:

`products` table:

id	name	category_id
1	Product 1	1
2	Product 2	1
3	Product 3	3

`categories` table:

id	name
1	Category 1
2	Category 2
3	Category 3

If the `category_id` is a foreign key (with `on delete cascade`) in the `products` table you have the following restrictions:

- You cannot insert a product with `category_id 31` since 31 is not present in the `categories` table.
- If you delete `Category 1` then `Product 1` and `Product 2` will also be deleted from the `products` table.

In one sentence: you cannot have inconsistent references in your tables.

However, there's also a dark side to foreign keys:

- A single delete query can start a whole chain of delete statements which can be dangerous.
- Sometimes (for example, when migrating data from an old structure to a new one) they can be annoying/unnecessary.
- Some people think that these cascading deletes and invalid references are the responsibility of your application, not your database. You can take care of data inconsistency without using foreign keys completely. Just think about Laravel model events and observers. They can do everything that foreign keys can. And they can be tested as well.
- Harder to scale. As far as I know, you cannot shard a database with foreign keys. This is why some cloud database providers such as Planetscale simply ban the usage of foreign keys. You can read more about it [here](#).

So using foreign keys can be (or should be) a choice. For example, I don't use them in solo projects, but I use them in my current team. The reasoning is this:

- I'm 100% sure that in a project where I'm the only developer I'll take care of everything without foreign keys. I'll also test these use cases. These projects are also **much smaller** than a typical company project.
- I'm 100% sure that we'll forget about these (without foreign keys) in a company environment where there are 50 new todos and deadlines every day.

The takeaway is this: if you're using foreign keys, you should use them in every scenario, so your database will maintain close to 100% data consistency in 99% of scenarios. Which is a pretty good thing. Especially in a larger project with 100s of tables.

Missing authorization in requests

Laravel HTTP requests is a pretty cool idea. They can provide us an easy to do:

- Validations
- Authorization
- And pretty basic data lookups

```
class StoreEmployeeRequest extends FormRequest
{
    public function getDepartment(): Department
    {
        return Department::where('uuid', $this->department_id)
            ->firstOrFail();
    }

    /**
     * @return Collection<Tag>
     */
    public function getTags(): Collection
    {
        return Tag::findOrFail($this->tag_ids);
    }

    public function rules()
    {
        return [
            'full_name' => ['required', 'string'],
            'email' => [
                'required',
                'email',
            ],
        ];
    }
}
```



```

        Rule::unique('employees', 'email')
            →ignore($this→employee),
    ],
    'department_id' => [
        'required',
        'string',
        'exists:departments,uuid',
    ],
    'job_title' => ['required', 'string'],
    'tag_ids' => ['sometimes', 'array'],
    'tag_ids.*' => ['exists:tags,id'],
];
}
}

```

By basic data lookups I mean the

- `getDepartment`
- and `getTags` methods

I know these database queries are not part of the HTTP-layer so instead of starting a religious war, let me explain: I do these getters only in smaller projects where I don't have different layers but only controllers, models and maybe some service classes. These getters make the controller (or service) much cleaner. However, if this doesn't fit your project/team/religious just ignore these methods please :)

Back to the request. Can you spot the missing part? Yeah, it's the authorization. Usually this is the most boring part so it's the easiest to forget about:

```
class StoreEmployeeRequest extends FormRequest
{
    public function authorize()
    {
        return $this->user()->can('employee.upsert');
    }
}
```

Why is this so important? Well, if authorization is important in your project then leaving out the `authorize` method is an obvious mistake/security issue.

Of course, you can always do something like that:

```
class EmployeeController
{
    public function store(Request $request)
    {
        if (!$request->user()->can('employee.upsert')) {
            abort(401);
        }
    }
}
```

However, the first problem is that you need to remember this in the update method as well:

```
class EmployeeController
{
    public function store(Request $request)
    {
        if (!$request->user()->can('employee.upsert')) {
            abort(401);
        }
    }

    public function update(Request $request)
    {
        if (!$request->user()->can('employee.upsert')) {
            abort(401);
        }
    }
}
```

If you use invokable controllers it's even easier to forget about it. And of course, authorizing in the controller means it does one more thing. And in my experience, as the project grows controllers usually always do "just one more thing" and never "one less thing." So if you do it in the controllers you add a little bit of complexity every time.

But usually, when you're using requests you can handle both the POST and the PUT (or PATCH) request in one class. I usually use the `Upsert` prefix so the class name will be `UpsertEmployeeRequest` and it takes of creating and updating an employee:

```

class UpsertEmployeeRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('employee.upsert');
    }

    public function getDepartment(): Department
    {
        return Department::where('uuid', $this->department_id)
            ->firstOrFail();
    }

    /**
     * @return Collection<Tag>
     */
    public function getTags(): Collection
    {
        return Tag::findOrFail($this->tag_ids);
    }

    public function rules()
    {
        return [
            'full_name' => ['required', 'string'],
            'email' => [
                'required',
                'email',
                Rule::unique('employees', 'email')
                    ->ignore($this->employee),
            ],
        ];
    }
}

```

```
],  
  'department_id' => [  
    'required',  
    'string',  
    'exists:departments,uuid',  
  ],  
  'job_title' => ['required', 'string'],  
  'tag_ids' => ['sometimes', 'array'],  
  'tag_ids.*' => ['exists:tags,id'],  
];  
}  
}
```

Of course you can use policies to achieve the same thing. I think it's a question of personal preference.

The point is: forgetting about authorization it's quite easy, so find a way to do it, and enforce it in every part of the project.

Validating in controllers

Alright, it's not a "mistake", but similar to authorization it adds some complexity to your controllers. And once again, after the `User.php` controller are next in line to become "legacy-looking" classes.

Consider this:

```
class EmployeeController
{
    public function store(Request $request)
    {
        if (!$request->user()->can('employee.upsert')) {
            abort(401);
        }

        $validated = $request->validate([
            'full_name' => ['required', 'string'],
            'email' => [
                'required',
                'email',
                Rule::unique('employees', 'email')
                    ->ignore($this->employee),
            ],
            'department_id' => [
                'required',
                'string',
                'exists:departments,uuid',
            ],
            'job_title' => ['required', 'string'],
            'tag_ids' => ['sometimes', 'array'],
```

```

        'tag_ids.*' => ['exists:tags,id'],
    ]);

    $employee = Employee::create($validated);

    $employee->tags()->attach($validated['tag_ids']);

    return EmployeeResource::make($employee);
}
}

```

And now this:

```

class EmployeeController
{
    public function store(StoreEmployeeRequest $request)
    {
        $employee = Employee::create($request->validated());

        $employee->tags()->attach($validated['tag_ids']);

        return EmployeeResource::make($employee);
    }
}

```

We can remove ~60% of the code by just adding a request class. If you have only 100 tables in your databases you have ~100 resource controllers. Imagine the amount of code that can be removed from these controllers...

This is why I try to avoid both validation and authorization in controllers. Also if you check out the [Laravel Daily channel](#) you can see that most junior code review videos start with these two things.

Incorrect dependencies

There are different layers in every Laravel application. Layers such as: HTTP, Business Logic, Database, etc. Each layer has its own dependencies. For example, the database layer should not depend on the HTTP layer. If it does, it's probably a poor design decision and you'll regret it later.

In my experience, the common one is when a model or model trait depends on an HTTP request or resource. The developer thinks that "I need to call this method from the controller and it'll use 3 parameters from the request. I just pass the whole request object so it's going to be simpler." But now, you cannot reuse this function from anywhere outside of a controller since it needs a request. You cannot call this method from a command, or a job, for example. You cannot schedule it either.

These are some examples I consider "incorrect" dependencies:

This class	Depends on these
Model	HTTP, Job, Command
Job	HTTP
Command	HTTP
Mail/Notification	HTTP, Job, Command
Service	HTTP
Repository	HTTP, Job, Command

As you can see, I really hate when things depend on HTTP-related classes 🙄

Disclaimer: if you're yoloing on an indie project and you want to ship it in 2 weeks, you should probably just ignore this tip.

Complex data objects

There are some typical classes that should not contain too much business logic since their main purpose is to hold data. These classes are:

- Resources
- Requests
- DataTransferObjects (DTO)
- Value Objects
- Mail
- Notification

As you may agree the purpose of these classes is to describe "something" and/or transfer it from one place to another. Such as a resource describing a model, and transferring it from the BE to the API. This is why I think it's important to keep these classes as simple as possible. The simpler your resource (or anything above) class is, the more you can reuse it later, for example.

In my experience, it's tempting to some developers to write queries or handle some business rules in resources, for example. Any time you want to do something like that, I think there's a better place for that code. My general rule of thumb is this: if this code does not fit any existing class, maybe it should go to a new service. Of course it depends on the current project, etc. The only exception for me, is one-liner `Model::find` queries in requests (you can see examples of that in this ebook).

Reversed if statements

I left out most of the clean code stuff from this ebook because there is endless amount of content based on `if` statements and `for` loops. However, there's one clean code issue in particular that is being made even by experienced senior devs.

Consider this method:

```
public function store(Request $request, PlanService
$planService)
{
    if ($planService->isEligible('repositories.enable', $user)) {
        EnabledRepository::create([
            'name' => $request->name,
            'github_id' => $request->github_id,
            'user_id' => $request->user()->id,
        ]);

        $this->emailProvider->updateSubscriber($request->user());

        return redirect(route('repositories'));
    } else {
        return redirect(route('subscription'));
    }
}
```

The actual code doesn't matter, it's just an example. The issue here, in my opinion, is that you take the "happy" path and put it inside an `if` statement. You're using a totally unnecessary indentation level. I always avoid code like this, and you do this instead:

```
public function store(
```

```
Request $request,  
PlanService $planService  
) {  
    if (!$planService->isEligible('repositories.enable', $user))  
    {  
        return redirect(route('subscription'));  
    }  
  
    EnabledRepository::create([  
        'name' => $request->name,  
        'github_id' => $request->github_id,  
        'user_id' => $request->user()->id,  
    ]);  
  
    $this->emailProvider->updateSubscriber($request->user());  
  
    return redirect(route('repositories'));  
}
```

Now the method has only one indentation level, and the `if` statement contains a **clear edge-case**. A case when the code should not run. And this is pretty important, because it makes a huge difference. You can write functions like this:

```
public function foo(): void
{
    if ($edgeCase1) {
        return;
    }

    if ($edgeCase2) {
        return;
    }

    if ($edgeCase3) {
        return;
    }

    $happyPath();
}
```

This function almost reads like a user story with the according acceptance criteria. If we go back to the previous example, this is what a user story looks like:

Enable a GitHub repository

As a user I can enable any of my GitHub repository. After that, I want to see my new repo on the repositories page.

Acceptance Criteria

- ☐ With the appropriate plan I can enable a new repository (AC1).
 - ☐ I can see the new repo on the repositories page (AC2).
 - ☐ The "Number of repositories" field in ConvertKit should be updated (AC3).
 - ☐ If I'm not eligible to enable a repo according to my current plan, I should see the subscription page (AC4).
-

The last AC is the only edge-case. Now let's pair the AC items with the code:

```
public function store(
    Request $request,
    PlanService $planService
) {
    // AC4
    if (!$planService->isEligible('repositories.enable', $user))
    {
        return redirect(route('subscription'));
    }

    // AC1
    EnabledRepository::create([
        'name' => $request->name,
        'github_id' => $request->github_id,
        'user_id' => $request->user()->id,
    ]);

    // AC3
    $this->emailProvider->updateSubscriber($request->user());

    // AC2
    return redirect(route('repositories'));
}
```

I think it look **much** better.

Other Issues

And finally here are some other issues that can cause trouble in your production system.

- Forgotten cache keys. Anytime a PR contains `Cache::remember` I'd like to check if the cache key is cleared when necessary. It can cause some hard to debug bugs.
- Environment checks such `App::environment() === 'production'` Of course, sometimes it's necessary but sometimes it's a smell. For example, junior devs sometimes don't know how to mock services in tests, so they check the environment in the controller and only dispatch jobs (for example), when it's not testing.
- Lack of database transactions. I always recommend to use transactions when you need to run multiple `insert`, `update`, or `delete` queries. One of the most common examples is when you insert a new record and then attach some n-n or 1-n relationships after that.
- Lack of error tracking. You cannot imagine how easier your life would be if you used Rollbar, or Sentry to track your errors instead of scanning 1000 lines long log files in the storage directory. It's a must-have in my opinion.
- Anything that goes against 12factor.
- Missing tests. If you say "I don't have time to write tests" I'd say "Then you're working for the wrong company and you should leave."

Thank you very much for reading this short ebook! I hope you have learned a few cool techniques.

The good news is that you don't have to remember these. I launched a code review tool called Laracheck that can analyze your GitHub PRs and will warn you if you made any of these issues. Try it out if you're interested in clean code.